Creative Variations of Monte Carlo Tree Search for a Minecraft Sword PvP Agent

Nguyen Huy An Khanh

COMP2050 Programming Project
Instructor: Leandro S. Marcolino
June 21, 2026

Abstract

This project studies Monte Carlo Tree Search (MCTS) for real-time sword combat in a Minecraft-like environment. The agent is implemented in TypeScript with Mineflayer for server interaction and prismarine-physics for a repeatable local duel simulator. Beyond a standard MCTS controller, the project implements and evaluates several variations: hybrid MCTS with a heuristic action bonus, risk-aware MCTS, high-exploration MCTS, and higher-budget MCTS. The final experiment generator produced 17 duel runs and 1,878 logged decisions, including 15 repeated strategy trials and two stress-test scenarios. The results show that the planner learns plausible combat behavior, especially closing distance before attacking, but also that stronger search settings can matter more than a hand-designed heuristic bonus in this simulator.

1 Introduction

Real-time game combat is a useful setting for studying artificial intelligence because an agent must make decisions under time pressure while handling movement, positioning, and uncertainty about the opponent. Minecraft sword PvP is a good example: a competent fighter must approach, maintain useful spacing, wait for attack range, and avoid wasting attacks during cooldown.

The goal of this project is to build and evaluate a sword-duel agent. The base algorithm is Monte Carlo Tree Search, a planning method that estimates action values by simulating possible futures. The project then goes beyond standard MCTS by adding domain-specific variations and comparing their effects experimentally.

The research questions are:

1. Can a small MCTS planner produce plausible Minecraft sword-duel behavior in a physics simulator?
2. How do creative variations such as heuristic evaluation, risk-aware scoring, exploration control, and larger search budgets change the agent's behavior?
3. Which measurements best explain the observed combat outcomes?

2 Related Work

MCTS is widely used in game AI because it can choose actions without exhaustively searching the full game tree. The algorithm repeats selection, expansion, simulation, and backpropagation.

Upper Confidence Bounds for Trees (UCT) balances exploiting actions with high current value against exploring less-visited actions. MCTS is best known from board games such as Go. AlphaGo later showed that tree search can be even stronger when guided by learned policy and value models, although this project uses a much smaller hand-designed evaluation because training deep networks is outside the project scope. The same planning idea can be adapted to real-time games by limiting the number of search iterations per decision.

Minecraft bot development commonly uses Mineflayer, a JavaScript API that exposes player entities, inventory, movement controls, chat, and attack actions. Prismarine-physics approximates Minecraft movement and collision behavior locally. In this project, these libraries provide the environment layer, while the combat search logic is implemented separately.

The project differs from standard turn-based examples because the action space is built from Minecraft controls: moving forward, sprinting, jumping, strafing, retreating, and attacking. MCTS therefore searches over short continuous-position futures generated by a physics simulator, while still choosing from a discrete action set.

3 Methodology

3.1 Environment

The implementation is written in TypeScript. The main runtime entry point is `src/bot.ts`; shared combat logic is in `src/combat.ts`; and report data generation is in `src/report-materials.ts`. The project supports two modes:

- **Server mode**: a Mineflayer bot connects to a Minecraft server, equips a sword, identifies an opponent, and applies the selected movement controls and attacks.
- **Physics mode**: two local simulated fighters duel in a flat prismarine-physics arena. This mode is used for repeatable experiments.

Each simulated fighter stores position, velocity, yaw, pitch, health, attack cooldown, and control state. The local simulator is not a complete Minecraft PvP clone, but it is useful for testing planning logic quickly and consistently.

3.2 Baseline and Standard MCTS

The baseline non-search behavior is a distance-threshold policy: move forward when far from the opponent, sprint when very far, jump for vertical gaps, and attack when inside sword reach. This policy is useful as a rollout opponent and as a conceptual baseline because it has no explicit planning.

The standard MCTS planner clones the current duel state and searches over candidate combat actions. Each decision performs:

1. **Selection**: choose a child with UCT.
2. **Expansion**: add an untried combat action.
3. **Simulation**: apply movement with prismarine-physics and resolve attacks when allowed.
4. **Rollout**: continue for a short horizon using heuristic actions.
5. **Backpropagation**: add the rollout score to all nodes on the selected path.

The reward function values survival, opponent damage, useful spacing, cooldown timing, and attack windows. It penalizes death, large vertical mismatch, and being too far from relevant combat range.

3.3 Creative Variations

Four MCTS variations were implemented for comparison:

- **Hybrid MCTS + heuristic evaluation**: adds a heuristic action bonus to the rollout score. The bonus rewards actions that approach at long range, attack when a valid cooldown window exists, and remain near useful melee spacing.
- **Risk-aware MCTS**: increases the value of safer spacing when the agent has lower health than the opponent.
- **High-exploration MCTS**: raises the UCT exploration constant, encouraging the search to test more candidate actions before committing.
- **Higher-budget MCTS**: increases iteration count and rollout depth, giving the planner more simulated futures before selecting a move.

These variations are intentionally not guaranteed to improve the result. They are designed to test different hypotheses about what matters in this combat domain: tactical heuristics, defensive survival, broader action exploration, and raw planning budget.

4 Experimental Evaluation

The evaluation uses physics mode because it is repeatable and fast enough for multiple runs. The report generator creates five strategy variants with three repeated trials each, using small deterministic jitter in the starting positions. It also creates two stress-test scenarios: a wide-arena opening and a low-health opening.

Each logged decision records the selected action, attack flag, rollout score, visit count, distance, health difference, cooldowns, positions, and movement controls. The generated files are stored in `debug/report/`. The most important files are `aggregate-summary.csv`, `summary.csv`, `episodes.csv`, `action-counts.csv`, and `tick-series.csv`.

The measurements are winner, final health, fight length, first attack tick, number of attack decisions, average distance, and average MCTS visits. Repeated trials make it possible to report means and variances rather than relying on a single duel.

5 Results

Table 1 summarizes the repeated strategy trials. Higher-budget MCTS and high-exploration MCTS won all three trials for Alpha in this experiment. Standard MCTS won two of three. The hybrid heuristic version did not improve the win rate here, despite closing distance slightly earlier on average. This is a useful negative result: the heuristic bonus changed behavior, but it did not automatically make the agent stronger.

Table 2 shows individual stress-test runs. The wide-arena example delays the first attack until tick 20 because the fighters start farther apart. The low-health example is won by Bravo, which shows that the simulator is not hard-coded to make Alpha win.

Table 1: Aggregate results from three repeated trials per strategy variant.

Variant	Runs	Alpha Win Rate	Mean Ticks	Var. Ticks	Mean First Attack	Mean Visits
Standard MCTS	3	0.667	58.000	0.667	14.000	15.397
Hybrid MCTS + heuristic evaluation	3	0.000	57.333	0.222	13.333	16.274
Risk-aware MCTS	3	0.333	56.000	0.667	12.000	15.473
High-exploration MCTS	3	1.000	57.667	0.889	13.667	14.986
Higher-budget MCTS	3	1.000	56.000	0.667	12.000	19.203

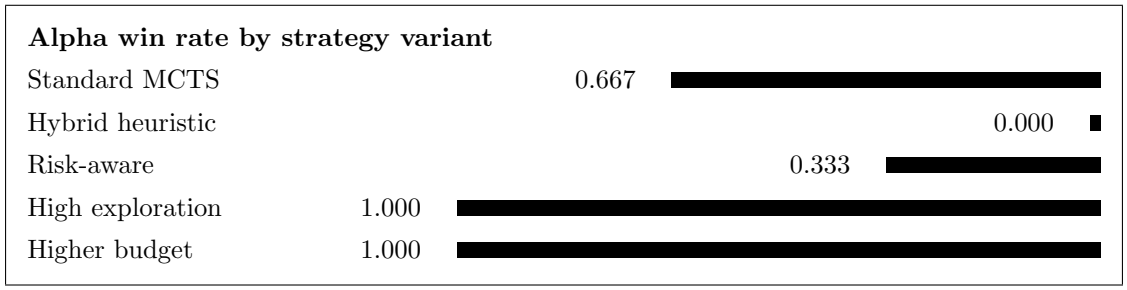

Figure 1: In-report bar chart of Alpha win rate across repeated strategy trials.

Across all runs, movement actions dominate early decisions. The most common action remains `sprint-advance`, which is expected because most duels begin outside melee range. Attack actions appear only after the fighters have moved into sword reach and cooldown conditions allow damage. This supports the claim that the planner is coordinating approach and attack timing rather than simply attacking every tick.

6 Discussion

The results suggest that search budget and exploration can matter more than a hand-designed heuristic bonus. Higher-budget MCTS achieved the highest average visit count and won all repeated trials. High-exploration MCTS also won all repeated trials, suggesting that trying a broader set of actions helps in the jittered openings.

The hybrid heuristic variation is more subtle. It reached first contact slightly earlier than standard MCTS, but it lost all three repeated trials. A likely explanation is that the heuristic bonus made the agent overvalue immediate attack windows or close spacing, while the opponent's response punished that positioning. This is a good example of reward shaping risk: a plausible hand-written heuristic can produce worse strategic behavior if it is not aligned with long-term survival.

The risk-aware variation won one of three repeated trials and produced the earliest mean first attack tick. In the low-health stress test, however, Alpha still lost. This indicates that defensive spacing alone is not enough when the health and damage disadvantage is large. More realistic defensive behavior would need knockback, shield timing, terrain use, or a better opponent model.

The main limitation is simulator fidelity. The local prismarine-physics duel approximates movement and cooldowns, but real Minecraft combat includes latency, knockback, shields, item effects, terrain, server version differences, and human unpredictability. Another limitation is sample size: three repeated trials per variant is enough to study mean and variance at a basic level, but not enough for strong statistical claims. A larger study should run dozens or hundreds of randomized openings per variant.

Table 2: Stress-test examples from the generated summary.

Scenario	Winner	Episodes	Ticks	Attacks	Alpha HP	First Attack
Wide arena example	Alpha	107	54	8	8	20
Low-health example	Bravo	70	35	6	0	13

7 Conclusion

This project implements a Minecraft sword PvP agent based on Monte Carlo Tree Search and evaluates several creative variations of the algorithm. The final system supports Mineflayer server play and local prismarine-physics experiments. The report data includes 17 duel runs and 1,878 logged decisions.

The experiments show that MCTS can produce plausible combat behavior in this environment: the agent closes distance, waits until melee range, and attacks in cooldown-limited bursts. The variations also show that changing the search procedure changes outcomes. In this experiment, high-exploration and higher-budget MCTS performed best, while the hybrid heuristic bonus changed behavior without improving win rate. Future work should use larger randomized batches, stronger opponent models, and a more faithful PvP simulator with knockback, shields, terrain, and item differences.

Acknowledgements

No external human collaboration was used for the implementation unless otherwise stated by the author. AI assistance was used to help inspect the brief, improve code structure, generate experiment material, and improve the readability of the report. The project uses open-source Minecraft bot libraries including Mineflayer, prismarine-physics, minecraft-data, prismarine-block, and vec3.

AI and Implementation Statement Summary

A separate `statement.pdf` is included with the submission. It explains which parts of the work were based on existing libraries, which parts were created or edited with AI assistance, and which parts represent the student's own implementation and design work.

References

[1] R. Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *Computers and Games*, Lecture Notes in Computer Science, vol. 4630, pp. 72–83, 2007. `https://doi.org/10.1007/978-3-540-75538-8_7`.

[2] L. Kocsis and C. Szepesvari. Bandit based Monte-Carlo planning. In *Machine Learning: ECML 2006*, Lecture Notes in Computer Science, vol. 4212, pp. 282–293, 2006. `https://doi.org/10.1007/11871842_29`.

[3] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012. `https://doi.org/10.1109/TCIAIG.2012.2186810`.

[4] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, and others. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016. https://doi.org/10.1038/nature16961.

[5] Mineflayer contributors. Mineflayer: Create Minecraft bots with a powerful, stable, and high level JavaScript API. GitHub repository, PrismarineJS. https://github.com/PrismarineJS/mineflayer.

[6] PrismarineJS contributors. prismarine-physics. GitHub repository, PrismarineJS. https://github.com/PrismarineJS/prismarine-physics.